Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Ingeniería del Software

Práctica #8

1. Prueba Unitaria

Las Pruebas Unitarias prueban unidades de programa individuales (clase, objeto o método), para verificar el funcionamiento correcto de las mismas con base en los requisitos. Es un proceso en el que se prueba la unidad funcional de código más pequeña.

Una prueba unitaria es un bloque de código que verifica la precisión de un bloque más pequeño y aislado de código de aplicación, normalmente una función o un método. Está diseñada para verificar que el bloque de código se ejecuta según lo esperado, de acuerdo con la lógica teórica del desarrollador

Estas pruebas tienen varias ventajas:

- Permiten la detección eficaz de errores: si hay errores de entrada, salida o basados en la lógica dentro de un bloque de código, las pruebas unitarias ayudan a detectarlos antes de que los errores lleguen a producción.
- Permiten tener pruebas de regresión: cuando cambia el código, se ejecuta el mismo conjunto de pruebas unitarias, junto con otras pruebas como las de integración, y se esperan los mismos resultados. Si las pruebas fallan indica errores basados en la regresión.
- Son documentación: las pruebas unitarias también actúan como una forma de documentación del código.
- Son fáciles de mantener y robustas

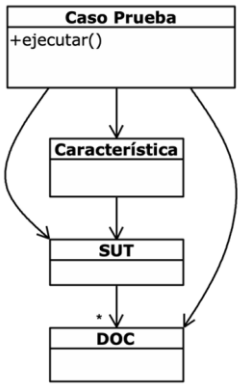
Hacer pruebas unitarias involucra escribir manualmente el código de pruebas que ejercita al sujeto bajo prueba en el propio lenguaje de programación de la aplicación, por eso requieren la habilidad de programar.

Un caso de prueba unitaria está formado por:

- Una característica (Feature)
- Un sujeto bajo prueba (SUT, por sus siglas en inglés)
- Un componente del que se depende (DOC, por sus siglas en inglés)

El SUT es un método de una clase que implementa la característica que estamos probando. El DOC son partes de la aplicación que no estamos verificando en una prueba unitaria en particular, pero de las que depende el SUT.

Un *Feature* es la unidad de funcionalidad comprobable que puede ser construida en la evolución de un programa informático. Una prueba tiene que ser algo muy concreto, a tal entrada tal salida, no puede ser algo muy general, esto es la característica (la unidad).



El SUT es un bloque de código que implementa la característica que estamos probando. La característica siempre se implementa en un código, eso es el SUT. Ciertas funcionalidades están definidas en requisitos, diseñadas en UML o implementada en código, por lo tanto el SUT es eso que soporta esa funcionalidad.

El DOC son partes de la aplicación que no estamos verificando en una prueba unitaria en particular, pero de las que depende el SUT. El DOC es un código que no quiero probar y voy a necesitar un doble, dummy o mock para poder hacer pruebas. Por ejemplo, objetos de otra clase al que el SUT envía mensajes o código que ataca la BD.

Una prueba unitaria debe ser ejecutada. Su ejecución puede tener dos resultados: Éxito (verde), Fracaso (Rojo).

2. Pruebas caja negra y caja blanca

Para diseñar pruebas unitarias utilizaremos dos tipos de técnicas: las pruebas caja negra o pruebas de especificación que verifican el comportamiento de la unidad observable externamente. Las pruebas caja blanca o pruebas de estructura que verifican la implementación interna de la unidad.

Las pruebas caja negra se realizan para verificar el comportamiento del componente sin tener en cuenta cómo se implementa dicho comportamiento. Estas pruebas tienen en cuenta la salida que el componente devolverá cuando se le da una entrada determinada en un estado determinado. Como el número de combinaciones posibles de entradas es considerable, se utilizan las técnicas de clases de equivalencia, valores límites y vector ortogonal para reducir los casos de pruebas a implementar. Una clase de equivalencia es un conjunto de valores de entrada para los que un objeto se comporta de forma similar. Entonces, el número de entradas se divide en clases de equivalencia y se prueba un componente con un valor de cada clase de equivalencia. Esto permite obtener casi la misma cobertura "efectiva" de prueba que obtendrías probando con cada valor, pero con un esfuerzo considerablemente menor.

En el caso de los valores límites, es una técnica que se aplica sobre la partición de clases de equivalencia cuando sus valores tienen un orden total, o sea, que para toda pareja de valores distintos se puede comprobar si uno es mayor que otro o viceversa. Por ejemplo, enteros, reales, caracteres por su código, cadenas de caracteres por su orden lexicográfico, enumerados por su ordinal, fechas, horas. Si estamos en presencia de esto, entonces, debemos

escoger como datos de entrada para la prueba los valores que estén en los límites de cada clase de equivalencia. Esta técnica agrega casos de prueba a la técnica de clases de equivalencia.

La técnica de vector ortogonal, también llamada “todos los pares”, nos sirve cuando hay dependencias entre las variables de entrada. Ella permite reducir el conjunto de los casos de prueba, seleccionando algunas combinaciones representativas que aseguren una cierta garantía de cobertura. Para seleccionar los casos de prueba, esta técnica te indica qué: 1) listes todas las combinaciones posibles entre los valores de las variables de entrada que son dependientes. 2) listes todas las combinaciones posibles entre todos los valores de pares de variables, vas combinando de dos en dos los distintos valores de las variables hasta tener todas las posibles combinaciones. 3) Escojas de la lista del paso 1 aquellos casos de pruebas que tengan más pares de la segunda lista, hasta tener todos los pares de la segunda lista en algún caso de prueba.

Las pruebas caja blanca por su parte verifican que un componente funciona internamente como se esperaba. Con estas pruebas nos aseguramos de probar todo el código, dado que cada sentencia y cada camino del código se ejecutará al menos una vez. Para realizar estas pruebas tenemos varias técnicas, una de las más sencillas es la del camino básico.

La técnica del camino básico, también llamada técnica de Grafo de Causa/Efecto, permite obtener una medida de la complejidad lógica de un diseño procedimental y utilizar esa medida como guía para la definición de un conjunto básico de caminos de ejecución. Para seleccionar los casos de prueba, esta técnica te indica qué: 1) crees el grafo de flujo de control, 2) calcules la complejidad ciclomática, 3) obtengas el número de caminos según la complejidad ciclomática y 4) saques los datos de pruebas necesarios para recorrer cada camino.

3. Motores de prueba

Los motores de búsqueda permiten crear pruebas que se lean de manera humana, por eso tienen métodos propios con nombres que le dan semántica a los test (assertEquals, assertArrayEquals, assertNull, assertTrue). Estos motores funcionan buscando todas las clases que tienen test y las ejecutan, y muestran un menaje si no se cumple la condición (verde, rojo).

Esto motores te permiten hacer aserciones. Una aserción (assert) es dar por hecho que algo esta funcionando y si no esta funcionando, entonces sacarlo a la luz. Un método Assert, es un método al que le pasas una condición y si no es verdad imprime un texto dando detalles.

En Java el motor de pruebas más conocido es JUnit. JUnit es un conjunto de clases (framework) que permite realizar la ejecución de clases Java de manera controlada, para poder evaluar si el funcionamiento de cada uno de los métodos de la clase se comporta como se espera.

JUnit es también un medio de controlar las pruebas de regresión, necesarias cuando una parte del código ha sido modificado y se desea ver que el nuevo código cumple con los requerimientos anteriores y que no se ha alterado su funcionalidad después de la nueva modificación.

Para usar JUnit en Visual Code se debe instalar la extensión: Test Runner por Java. Más información se puede encontrar en: https://code.visualstudio.com/docs/java/java-testing

4. Implementar una prueba unitaria

Luego de definir los casos de prueba al aplicar las técnicas caja negra y caja blanca, el siguiente paso es implementarlas. Para implementarlas debes estructurar la prueba siguiendo el patrón AAA: Arrage, Act y Assert.

El Arrange va primero y prepara la prueba unitaria inicializando o creando los objetos que se necesiten. Lugo viene el Act que ejercita o corre el SUT. Por último, le hace el Assert que es responsalbe de verificar la prueba unitaria, comprobar que ha ido bien.

Esto se programa según el motor de búsqueda que se use. En el caso de JUnit una prueba unitaria con el patrón AAA se vería como:

```
import org.junit.Test;

public class CalculatorTest {
     @Test
     void testAdd() {
          //Arrange
          Calculator calculator = new Calculator();

          //Act
          int result = calculator.add(2,3);

          //Assert
          assertThat(result, is((int) 5));
     }
}
```

5. Ejercicios

a) Dado el siguiente método sqrt de la clase Math que devuelve la raíz cuadrada positiva correctamente redondeada de un valor doble, diseñe los casos de prueba unitaria necesarios usando las técnicas caja negra de clases de equivalencia y valores límites. en cuenta los siguientes casos especiales: si el argumento es NaN o menor que cero, entonces el resultado es NaN, si el argumento es infinito positivo, entonces el resultado es infinito positivo, si el argumento es cero positivo o cero negativos, entonces el resultado es el mismo que el argumento, de lo contrario, el resultado es el valor doble más cercano a la verdadera raíz cuadrada matemática del valor del argumento. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
class Math {
       public static double sqrt(double value) {…}
}
```

b) Dado el siguiente método factorial de la clase Combinatorics que devuelve la factorial de un valor long mayor o igual que 0, implemente los casos de prueba unitaria necesarios usando las técnicas caja negra de clases de equivalencia y valores límites. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
class Combinatorics {
       public static long factorial(long value) {…}
}
```

c) Dado el siguiente método next de la clase Planet que devuelve el planeta siguiente de la lista de planetas del sistema solar, y si es el último planeta devuelve el primero, implemente los casos de prueba unitaria necesarios usando las técnicas caja negra de clases de equivalencia y valores límites. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
public enum Planet {
      MERCURY,
      VENUS,
      EARTH,
      MARS,
      JUPITER,
      SATURN,
      URANUS,
      NEPTUNE;
      public static Planet next(Planet planet){ … }
}
```

d) Dado el siguiente método includes de la clase ClosedInterval que devuelve verdad si el intervalo incluye otro intervalo dado, implemente los casos de prueba unitaria necesarios usando las técnicas caja negra de clases de equivalencia y valores límites. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
class ClosedInterval {
      private double min;
      private double max;
            .
            .
            .
}
```

e) Dada la siguiente clase Date, implemente los casos de prueba unitaria necesarios usando las técnicas caja negra de clases de equivalencia y valores límites, para probar el constructor.

Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
public class Date {

       private int day;
       private int month;
       private int year;

       private static final int[] DAY_LIMIT = {31, 28, 31,
                             30, 31, 30, 31, 31, 30, 31, 30, 31};

       private static final int MONTH_LIMIT = DAY_LIMIT.length;
                                 .
                                 .
                                 .
}
```

f) Dado el método next de la clase Date del ejercicio anterior, que devuelve el próximo día dado al día definido, implemente los casos de prueba unitaria necesarios usando las técnicas caja negra de clases de equivalencia y valores límites. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
public Date next() { .. }
```

g) En base a la implementación del método next del código de la clase Date del ejercicio “e”, implemente los casos de prueba unitaria necesarios usando la técnica caja blanca de camino básico. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

h) En base a la implementación del siguiente método search, que realiza una búsqueda binaria sobre un arreglo de datos y devuelve la posición en el arreglo si consigue el valor o “-1” en caso contrario, implemente los casos de prueba unitaria necesarios usando la técnica caja blanca de camino básico. Luego de diseñar, implemente los casos de pruebas con JUnit. Use como base el código que se anexa con esta práctica.

```
public static int search( int [] values, int value) {
   int start = 0;
   int end = values.length - 1;
   int pos;
   while (start <= end) {
      pos = (start + end) / 2;
      if (values[pos] == value){
         return pos;
      } else if (values[pos] < value) {
         start = pos+1;
      } else {
         end = pos-1;
      }
   }
   return -1;
}
```